

The HapBlockR Breeder's Guide

Choosing parents and crosses with haplotype-aware tools

Document ID: HBR-GUIDE-001

Edition: 6

Compatible package version: 0.3.12.9000

Release date: 26 July 2026

Author: Félicien Akohoue, PhD

Source: `inst/guide/HapBlockR_Breeder_Guide.md`

HapBlockR converts the breeder's data, objectives and constraints into traceable parent, cross and mating recommendations. Their relevance is driven principally by the quality and representativeness of the supplied data and by how accurately the specified parameters describe the programme. Keep the package result object with the signed decision record so that this basis remains explicit.

Contents

1. Purpose and scope
2. Before any recommendation
3. The nine decision tools and their variants
4. Worked crossing decision
5. Interpreting quality control and uncertainty
6. Interpreting results and defining their scope
7. Decision sign-off
8. Supporting tools and complete function map
9. References and change history

1. Purpose and scope

HapBlockR connects genomic evidence to a parent shortlist, cross ranking, mating plan, or diversity collection. The nine tools answer different questions. They are not a ladder in which the last tool is always best.

Decision need	Tool	Primary output
Transparent merit baseline	Truncation selection	Top candidates by a trusted score
Complementary block coverage	GA parent selection	A fixed-size parent set
Complementary coverage plus whole-genome merit	Joint GA+TS parent selection	A fixed-size parent set optimised for both signals
Representation across groups	Family or cluster quota	Group-balanced parent set
Promising individual crosses	Usefulness criterion	Cross mean, variance, downside risk
Gain under coancestry control	Optimal contribution selection	Contributions and mating plan
Visible gain-diversity trade-off	Pareto frontier	Non-dominated alternatives
Independent small-problem check	Exact validation	Optimality gap and feasibility
Diversity-first sampling	Core collection	A genetically dispersed subset

Truncation selection, coverage-only GA, joint GA+TS, and family or cluster quota selection are alternative shortlisting methods. Usefulness criterion and optimal contribution selection take a candidate set and answer different downstream questions. Pareto analysis exposes policy trade-offs. Exact validation is a quality-assurance tool. Core collection is appropriate when diversity itself is the objective.

2. Before any recommendation

2.1 Minimum evidence

- Define the target population of environments and the breeding objective.
- Confirm immutable candidate, sample, variant, study, and environment IDs.
- Confirm the target estimand, estimation basis, direction of improvement, uncertainty source, missing-data rules, and exclusions. Units are retained when supplied but are not mandatory.
- Match cross-validation to the intended use: random folds for interpolation within a population, grouped folds for new families or populations, and forward validation for future years, cycles or environments.
- Record the random seed, software versions, input hashes, transformations, warnings, fallback modes, and external-tool checksums.
- Require a validation status of pass before promoting an output to a recommendation.

2.2 Externally analysed target values

HapBlockR begins after the field-trial or genetic-evaluation analysis. It does not accept raw plot observations, inspect replicate, block, row or column factors, or fit the field design. The approved external analysis must produce one genotype-level estimate per trait and, when relevant, environment. HapBlockR then uses that estimate as the response in its internal marker, haplotype, block, multi-trait or genotype-by-environment model.

`prepare_breeding_targets()` separates two concepts that must not be confounded:

1. the **estimand** states what biological or statistical quantity is being supplied; and
2. the **estimation basis** states how that quantity was estimated.

For example, breeding value is an estimand. It may have been estimated with an identity or pedigree covariance model. Conversely, a Best Linear Unbiased Prediction (BLUP) describes an estimation method and must still be linked to the genetic quantity that was predicted.

input_type	Definition	Required uncertainty and treatment
"adjusted_mean"	A model-adjusted entry mean. This includes a Bayesian posterior adjusted mean when the analysis does not use the Best Linear Unbiased Estimate (BLUE)/BLUP distinction. It is not a raw arithmetic mean.	Supply standard error, posterior standard deviation, precision or full sampling covariance. The value is not deregressed.
"BLUE"	Best Linear Unbiased Estimate of an entry fitted as a fixed effect. It is not shrunk towards a population mean.	Supply standard error, precision or full sampling covariance. The value is not deregressed.
"BLUP_identity"	Random genotype or entry effect with covariance $I \times \text{genetic variance}$. The identity matrix describes the assumed covariance; it does not assert that the genotypes are biologically unrelated.	Supply reliability, or prediction error variance (PEV) and genetic variance. HapBlockR deregresses the prediction.
"PBLUP"	Pedigree BLUP of additive breeding value with covariance $A \times \text{additive genetic variance}$, where A is the numerator relationship matrix.	Supply reliability, or PEV and additive genetic variance, plus the complete named A matrix. HapBlockR deregresses the prediction.
"BV"	Additive, transmissible breeding value. This is an estimand rather than an estimation method.	Declare <code>estimation_basis = "fixed", "identity"</code> or <code>"pedigree"</code> and supply the corresponding uncertainty. An external genomic basis is rejected.
"GCA"	General combining ability in a defined tester or mate population.	Name the tester population and state whether GCA was fixed or random. Fixed GCA follows BLUE treatment; random GCA is deregressed.

input_type	Definition	Required uncertainty and treatment
"TGV"	Total genetic value, including additive and dominance components.	Supply additive and dominance values in separate columns, declare their estimation basis and supply the corresponding uncertainty.

Declared trait units are optional. However, the direction of improvement is mandatory for interpretation.

`lower_is_better` reverses a trait once during target preparation. Every downstream `model_value`, internal index and `merit_score` then uses one convention: larger is better.

For a standard error SE, raw precision is $1 / SE^2$. HapBlockR normalises this precision to a mean of one within the trait-analysis group before it enters the residual model. For an identity BLUP, `reliability = 1 - PEV / genetic_variance`. For a pedigree BLUP, `reliability = 1 - PEV / (A_ii × additive_genetic_variance)`. A standard error of difference is not a PEV and is rejected for deregression.

External genomic BLUPs, genomic estimated breeding values and external selection-index values are not accepted as target types. Accepting them would duplicate genomic information and obscure the origin of marker, block and haplotype effects. HapBlockR instead fits those effects internally and builds the selection index from the resulting trait predictions.

2.3 Reliability gate

HapBlockR reports reliability whenever the fitted model provides prediction error variance. Set a programme-specific minimum before seeing the final ranking. Candidates and crosses that meet the gate form the recommendation; those below it remain clearly identified for exploratory use. Where a method does not estimate uncertainty, the result records that fact rather than substituting an artificial value.

2.4 Operational feasibility

Before optimisation, assemble a candidate-status table covering:

- permitted female and male roles;
- flowering windows and synchrony;
- fertility and reciprocal-cross restrictions;
- seed or pollen availability;
- crossing capacity by parent and period;
- forbidden and required pairs;
- heterotic groups and subpopulation quotas;
- quarantine or movement restrictions; and
- minimum viable family size.

The final plan must include a feasibility certificate, excluded records, and binding constraints. A mathematically attractive but infeasible plan is not a breeding recommendation. Use `screen_candidate_crosses()` before ranking and `certify_mating_plan()` before sign-off.

3. The nine decision tools and their variants

This chapter describes the nine principal breeder-facing decision tools. For each tool, distinguish the biological question from the optimisation method. A technically more elaborate method is not automatically more appropriate. Use the simplest method that represents the programme's actual objective and constraints.

Each promoted result should answer five questions:

1. What candidate population and evidence release were analysed?
2. What objective, direction and constraints were declared before optimisation?
3. Which variant or engine was used, and why?
4. What recommendation did the package return automatically?
5. Which diagnostics, scope assumptions and reserve choices accompany it?

3.1 Truncation selection

Function: `truncation_selection()`

Question: Which candidates have the highest trusted merit score?

Appropriate use

Use truncation selection as the transparent baseline for every more complex method. It is often adequate when the breeding objective is represented by one validated score, candidates are not excessively related, no target haplotype must be recovered, and operational restrictions can be applied before ranking.

The input `score` is a named numeric vector. Its names are immutable candidate IDs. Every selection function uses one unambiguous direction: **larger values always mean greater breeding merit**. Reverse the sign of a lower-is-better trait, such as disease severity, maturity duration or plant height where reduction is desired, or assign it a negative selection-index weight before constructing `score`. Eligibility thresholds must be applied to this directional merit score, not to an untransformed raw phenotype. `n_founders` specifies the required shortlist size.

Eligibility-floor variants

`min_sel_value` is optional. `min_sel_mode` determines its meaning:

Variant	Interpretation	Example use
"value"	Retain candidates whose score is at least the stated value	A validated minimum disease-resistance index
"percentile"	Retain the stated fraction of candidates from the top of the directional score	<code>0.40</code> retains the best 40%
"sd_above_mean"	Retain candidates with directional merit at least the stated number of SD above the mean	<code>0</code> retains candidates at or above the mean; <code>1</code> requires at least one SD superiority
"relaxed_pool"	Deliberately broad-pool rule: retain candidates above $\text{mean}(\text{score}) - k \times \text{SD}(\text{score})$	Preserve a broader pool for haplotype complementarity or diversity

For routine selection of superior candidates on a scale that varies between cycles, use `"sd_above_mean"`. The `"relaxed_pool"` mode does not mean that low scores are preferred: all candidates are still ranked from highest to lowest. It exists for programmes that intentionally admit some below-population-average candidates into the search because they carry complementary favourable haplotypes or preserve useful diversity.

The floor defines eligibility; it does not change the order among eligible candidates. If fewer eligible candidates remain than `n_founders`, the function reports that fact and returns all candidates who passed the declared floor; it never weakens the threshold automatically.

Automatic recommendation and review

The package sorts eligible candidates by decreasing score and returns the top `n_founders` in `$selected`. There is no stochastic search and no need for replicated runs. Retain the effective cut-off and identify ties around the last selected position.

Review:

- score construction, units and direction;
- prediction reliability and validation accuracy;
- the eligibility rule and excluded candidates;
- family, genomic-cluster and relationship concentration;
- whether the last selected and first reserve candidate are practically distinguishable; and
- whether the result remains stable when uncertain weights or thresholds vary.

Move from truncation to family selection, GA selection or optimal contribution selection when the objective includes group representation, favourable-block complementarity, differential contributions or long-term coancestry.

```
ts <- truncation_selection(
  score = prediction$gebv,
  n_founders = 20,
  min_sel_value = 0.40,
  min_sel_mode = "percentile"
)
ts$selected
```

3.2 Coverage-only genetic-algorithm parent selection

Function: `select_parents_ga()`

Question: Which fixed-size set jointly covers favourable values across target blocks?

Required evidence and core objective

`value_matrix` contains candidates in rows and target blocks in columns. Entries normally represent local GEBV or another comparable favourable-value score. Row names and column names must identify candidates and blocks. `block_weights` can give greater influence to pre-declared priority blocks.

For a proposed founder set, the GA sums the best attainable value at every target block. It therefore optimises the set's complementarity, not merely the individual ranking of its members. Use `top_candidates` only as a documented computational pre-filter; an omitted candidate cannot be recovered by the GA.

Crossing-scheme variants

Optimal Haplotype Selection (OHS) seeks a complementary parent set in which distinct selected parents contribute the favourable haplotypes required across target segments. Optimal Population Value (OPV) evaluates the value available from the selected population, allowing one selected parent to supply the best state at a segment. `Haploid_OHS` is the haploid single-parent-donation formulation of OHS.

strategy	Block value used by the objective	Breeding interpretation
"no_selfing"	Mean of the two largest values from distinct selected parents	Two parents must jointly realise the block value
"OHS"	Same calculation as "no_selfing"	Haplotype-mode description in which distinct parents each contribute a copy
"selfing"	Largest value among selected parents	A parent may realise its value without a complementary parent
"OPV"	Same calculation as "selfing"	One founder carrying the favourable state is sufficient for the population
"Haploid_OHS"	Same calculation as "selfing"	A single heterozygous parent may donate non-homologous gametes

The last three strategies are mathematically identical under the present maximum-value formulation. Their names preserve different breeding interpretations; they are not three different numerical algorithms.

Objective variants

`select_parents_ga()` is deliberately the coverage-only tool. It accepts two optional relationship controls:

- `coancestry_weight > 0` subtracts a weighted mean pairwise relationship from the coverage objective.
- `target_degree` specifies the gain-to-diversity position on a 0–90 scale and derives a relationship ceiling from the supplied `G`.

Do not supply both controls in one call. Use `select_parents_ga_ts()` when whole-genome merit must influence eligibility or the continuous objective. Keeping the functions separate makes the declared breeding objective visible from the function name and prevents coverage-only and joint GA+TS analyses from being confused.

Replicated search and the automatic recommendation

The breeder does **not** run several calls and inspect every run manually. Both `select_parents_ga()` and `select_parents_ga_ts()` perform replicated optimisation:

- `n_reps = 5` by default;
- replicate seeds are derived reproducibly as `seed`, `seed + 1`, and so on;
- each replicate is repaired and checked for exact founder count and hard relationship feasibility;
- the complete objective is calculated for every replicate; and
- the feasible replicate with the greatest complete objective is selected automatically.

The returned `$selected`, `$fitness`, `$per_block`, `$ga_fit`, `$run_id` and `$seed` all belong to that automatically selected replicate. `$stability$best_rep` identifies it and `$stability$fitness_values` records the comparison. The implementation uses each tool's complete declared objective: coverage and any coancestry term for coverage-only GA, and coverage, merit and any coancestry term for joint GA+TS.

The breeder reviews the *summary diagnostics*, not every run:

- `$converged` reports whether the selected replicate reached the GA plateau rule before `maxiter`;
- `$stability$converged` reports this for all replicates;
- `$stability$fitness_range` shows whether independent searches reached materially different objective values;
- `$stability$selection_freq` shows how often each candidate occurred;

- `$stability$mean_relationship` and `$stability$mean_merit` show variation among replicate solutions; and
- `$feasible` and the result-contract quality gates confirm that the returned set satisfies hard postconditions.

High selection frequency is evidence that the same parents are repeatedly supported. Low frequency does not require the breeder to choose another run by eye. It indicates alternative near-equivalent founder sets or inadequate search stability. In that case, increase `popSize`, `maxiter`, `run` or `n_reps`, examine the objective and candidate evidence, and rerun the package. If the programme applies an additional rule after optimisation, record that rule and the resulting selection as a separate decision.

```
ga <- select_parents_ga(
  value_matrix = prediction$local_gebv[, target_blocks, drop = FALSE],
  n_founders = 20,
  strategy = "no_selfing",
  G = prediction$G,
  target_degree = 45,
  n_reps = 10,
  popSize = 200,
  maxiter = 400,
  run = 80,
  seed = 20260725
)

ga$selected                # automatic recommendation
ga$stability$best_rep      # replicate used for the recommendation
ga$stability$fitness_values # objective values from all searches
ga$stability$selection_freq # compact parent-stability diagnostic
```

Compare the automatic GA set with truncation selection and inspect genomic-cluster representation. The returned set is the best feasible result found for the declared objective and parameters; adding operational or biological requirements to those inputs brings them into the optimisation and validation record.

3.3 Joint GA+TS parent selection

Function: `select_parents_ga_ts()`

Question: Which fixed-size parent set provides complementary favourable-block coverage while also maintaining high whole-genome merit?

What GA+TS means

GA+TS is a joint objective, not a sequential procedure that truncates the population and then applies a separate GA. For every proposed parent set S , the tool evaluates:

$$\text{objective}(S) = \text{coverage}(S) + \lambda * \text{mean_merit}(S) - \gamma * \text{mean_relationship}(S)$$

The coverage term is the same strategy-specific OHS or OPV calculation used by `select_parents_ga()`. The merit term is the mean directional whole-genome score of the selected parents. The relationship term is included only when `coancestry_weight` is positive; `target_degree` may instead impose an internally calibrated relationship ceiling. Because every solution has exactly `n_founders`, maximising mean merit and maximising total merit produce the same ranking of candidate sets.

Merit input

`merit_score` is mandatory and must be a named finite numeric vector covering every candidate in `value_matrix`. Larger values must always represent greater breeding merit. Appropriate inputs include a HapBlockR GEBV or an internal Smith–Hazel, Pešek–Baker, desired-gain selection index (DGSI), or quadratic genomic selection index (QGSi) produced after the relevant internal modelling. Do not provide a raw phenotype or an externally calculated selection index directly at this selection stage.

Merit-control variants

Exactly one merit-control form is active:

Control	Meaning	Recommended use
<code>merit_priority</code>	Breeder-facing relative merit emphasis in $(0, 100]$; default 50	Routine analyses because HapBlockR calibrates the corresponding raw weight from the actual candidate pool
<code>merit_weight</code>	Positive raw multiplier on mean merit	Advanced analyses in which the programme has justified and documented the raw scale

`merit_priority = 100` scales the empirically attainable merit contrast to the attainable coverage contrast. A value of 50 gives the merit term half that reference span. It does not mean that 50% of the parents come from truncation selection. `suggest_merit_weight()` exposes the same calibration and returns the coverage span, merit span and resulting raw weight.

Zero is deliberately excluded from the hybrid function because it would silently remove merit from the objective. Use `select_parents_ga()` for coverage-only selection.

Eligibility-floor variants

`min_sel_value` is optional and defines a hard eligibility rule before the joint GA+TS search. It is distinct from the continuous merit term: the floor decides who may enter the search, while `merit_priority` or `merit_weight` continues to distinguish all eligible candidates.

<code>min_sel_mode</code>	Rule
"value"	Keep <code>merit_score >= min_sel_value</code>
"percentile"	Keep the stated fraction from the top of <code>merit_score</code>
"sd_above_mean"	Keep scores at least the stated number of SD above the mean; 0 retains candidates at or above the mean
"relaxed_pool"	Keep scores above <code>mean(score) - k × SD(score)</code> when the programme deliberately wants a broader pool

"relaxed_pool" does not reverse selection direction. Higher merit remains better throughout the objective. The deprecated name "sd_below_mean" maps to this broad-pool rule only for compatibility.

Relationship controls and automatic recommendation

The hybrid tool accepts the same `G`, `coancestry_weight` and `target_degree` controls as coverage-only GA. Do not combine `coancestry_weight` and `target_degree`. It also uses the same replicated search: all runs are evaluated automatically, and the best feasible complete-objective solution is returned.

`$objective_components` reports coverage, merit bonus, coancestry penalty and total objective separately.

`$mean_merit`, `$mean_relationship`, `$stability` and `$result_contract` provide the corresponding evidence and provenance.

```
ga_ts <- select_parents_ga_ts(
  value_matrix = prediction$local_gebv[, target_blocks, drop = FALSE],
  n_founders = 20,
  merit_score = prediction$gebv,
  strategy = "OHS",
  merit_priority = 50,
  min_sel_value = 0,
  min_sel_mode = "sd_above_mean",
  G = prediction$G,
  target_degree = 30,
  n_reps = 10,
  popSize = 200,
  maxiter = 400,
  run = 80,
  seed = 20260726
)

ga_ts$selected
ga_ts$objective_components
ga_ts$stability$best_rep
ga_ts$stability$selection_freq
```

3.4 Family or genetic-cluster quota selection

Function: `select_parents_by_family()`

Question: Which groups, and which candidates within them, should be represented?

Group-definition variants

group_by	Required evidence	Interpretation
"family"	Trustworthy family labels in <code>family</code>	Protects recorded pedigree groups
"genetic_cluster"	Genomic relationship matrix <code>G</code> and <code>n_clusters</code>	Defines groups from realised genomic similarity

Use family mode when the recorded pedigree is the programme's policy unit. Use genetic-cluster mode when labels are absent, uncertain, or less relevant than realised structure. Report those memberships as realised genetic clusters, while recorded families retain their pedigree interpretation.

Within-group selection variants

family_select_mode	Rule
"count"	Select <code>n_per_family</code> candidates from each retained group
"percentage"	Select the declared <code>pct_per_family</code> from each group
"sd_threshold"	Select candidates with directional merit at least $\text{mean}(\text{score}) + \text{sd_threshold} \times \text{SD}(\text{score})$, calculated across the eligible candidate population
"check_relative"	Select relative to a declared check candidate or check value and margin

For "sd_threshold", 0 retains candidates at or above the population mean and 1 requires at least one SD superiority. Negative thresholds deliberately broaden the eligible pool. As elsewhere, construct `score` so that larger always means better before applying this rule.

Group ranking can use "shrunk_topk_mean" or "topk_mean". Shrinkage is preferred when group sizes are unequal because a small group with one extreme observation should not automatically outrank a well-supported group. Variance estimation can use "rem1" or "anova", and `bias_correction = "order_stats"` reduces winner's-curse distortion in top-order summaries.

Haplotype-diversity variants

With `ensure_haplotype_diversity = TRUE`, the within-group choice can be adjusted using:

- "coverage_gain": prefer candidates that add favourable block coverage to the growing selected set; or
- "dominant_block": prefer complementary dominant haplotype states using supplied phased haplotypes.

`within_group_target_degree` and `G` can impose relationship control inside each group. This is not a substitute for a whole-plan OCS analysis.

Review group sizes, eligibility, shrinkage, the within-group rule and groups receiving no contribution. A quota deliberately protects representation; the reported group scores and uncertainty show the genetic evidence supporting each allocation.

3.5 Usefulness criterion

Function: `usefulness_criterion()`

Question: Which pairings combine expected progeny mean with useful segregation?

The usefulness criterion is:

$UC = \text{expected cross mean} + \text{selection intensity} \times \text{predicted progeny SD}.$

The function accepts either `parent_ids`, which generates all permitted pairs, or an explicit `cross_pairs` table, which is preferred after operational screening. `selected_proportion` determines selection intensity. When `n_progeny` is supplied, finite-family selection intensity is estimated from simulation rather than treated as an infinite-population approximation.

Variance-model variants

variance_mode 1	Required inputs	What is represented	Modelling scope
"block_independent"	<code>local_gebv</code> and target blocks	Sum of independent block segregation variances	Ignores linkage between blocks
"phased"	Phased haplotypes and effects	Exact within-block four-gamete enumeration	Blocks remain independent
"linked"	Phased haplotypes plus genetic map or LD information	Monte Carlo inheritance with block-to-block recombination	Accuracy depends on phase, map and simulation size
"simplemating"	Suitable genotype coding and SimpleMating inputs	Multi-locus progeny variance for RIL or DH designs	Optional dependency and method-specific data assumptions

For "linked", increase `n_sim_linked` until cross rankings are stable. `type = "RIL"` and "DH" represent different progeny-development systems; `generation` must match the proposed breeding stage. `het_to_na` should be used only when its genotype-coding assumption is appropriate.

Reliability and downside risk

Pass `gebv_se`, `gebv_reliability` and `phasing_reliability` whenever available. The function reports uncertainty, a downside quantile and recommendation reasons. `min_reliability` makes the programme's evidence threshold operational. Review:

- expected cross mean;
- predicted variance and its model;
- UC and the declared selection intensity;
- downside quantile;
- uncertainty interval;
- parental and phasing reliability;
- number of simulated progeny or inheritance replicates; and
- practical feasibility.

Promotion can therefore require a specified UC, downside-risk and reliability threshold. When phase is used in the variance model, its reported accuracy becomes part of that decision rule.

3.6 Optimal contribution selection

Function: `select_parents_ocs()`

Question: How much should each parent contribute while expected gain is balanced against coancestry?

Inputs are a named merit vector and a genomic relationship matrix `G`. `n_crosses`, `max_contrib_per_parent`, selfing and repeated-mating rules translate contributions into an operational mating plan. `target_degree` ranges from 0, which prioritises gain, to 90, which prioritises diversity.

Engine variants

engine	Role	Implementation and use
"auto"	Selects an available supported engine according to documented precedence	The result records the resolved engine
"optisel"	Solves optimal contributions and derives matings using optiSel	No external executable; re-solves on the retained subset when <code>n_parents_max</code> is active
"alphamate"	Uses the external AlphaMate optimisation programme	Requires an independently acquired executable and retained external provenance
"simplemating"	Selects discrete crosses through SimpleMating	Related cross-selection route, not mathematically identical to contribution-level OCS

AlphaMate enforces `n_parents_max` natively during allocation. The optiSel route retains the leading contributors and re-solves the OCS frontier and constrained optimum on that subset before constructing the mating plan. SimpleMating reports the control as unsupported because its discrete cross-selection interface has no corresponding argument. `rescale_nrm` controls relationship-matrix scaling; use the same convention across cycles.

Review:

- contributions are non-negative and sum to one;
- realised expected gain and coancestry;
- active and binding constraints;

- number of contributing parents;
- solver status and any fallback;
- integer offspring allocation;
- selfing and repeated-mating compliance; and
- the feasibility certificate for the resulting crossing plan.

Use OCS for recurrent programmes, population improvement and any decision in which both contribution amount and long-term inbreeding matter. For a small set of one-off exploratory crosses, the UC ranking may already provide the required cross-level decision.

3.7 Pareto frontier

Function: `select_parents_pareto()`

Question: Which founder sets are non-dominated for block coverage, merit and relationship?

The function reruns GA selection across `coancestry_weights`. Each grid point is itself a replicated stochastic search controlled by `n_reps`. The output contains all runs and a non-dominated frontier.

`pareto_front()` is the general supporting function that identifies non-dominated rows in a table of competing objectives. Use `select_parents_pareto()` for the complete breeder-facing GA sweep and use `pareto_front()` when an already constructed scenario table needs the same dominance calculation.

`strategy` has the same five variants as `select_parents_ga()`. `block_weights`, `top_candidates` and GA controls have the same meaning. When `merit` is supplied, the result reports the merit of each candidate solution; the principal sweep remains the coverage–coancestry trade-off.

The function returns the complete non-dominated frontier. Selection from that frontier is governed by the programme's declared gain, diversity and operational policy. A breeder should:

1. remove dominated and infeasible solutions;
2. reject solutions outside pre-declared gain or diversity limits;
3. compare the remaining points for stability and operational feasibility;
4. select one point using the programme's stated policy; and
5. retain at least one neighbouring frontier point as a reserve.

Select the point by applying the declared programme policy, then record the chosen `run_index`, parent set, neighbouring alternatives and reason for accepting the selected gain–risk balance.

3.8 Exact validation

Function: `validate_crosses_exact()`

Question: How close is a heuristic discrete cross plan to the best attainable plan under the same candidate set and constraints?

The function performs exact integer optimisation on a tractable candidate cross table. `criterion_col` identifies the quantity to maximise, normally UC. Relationship can be supplied as an existing cross column or calculated from G. `n_cross` fixes the number of selected crosses and `max_cross` limits the number assigned to one parent.

Validation variants

- With `heuristic_plan`, the result reports its objective and optimality gap against the exact solution.
- `culling_pairwise_k` restricts the candidate set before exact optimisation; it is a declared approximation and must be recorded.
- `max_vars` protects the exact solver from an intractably large candidate problem.

Exact validation establishes the best attainable plan for the supplied candidate set, criterion and constraints. A like-for-like comparison therefore keeps those inputs fixed. Phenotype quality, prediction accuracy, the UC model and practical restrictions enter through the candidate cross table and constraints supplied by the user.

3.9 Core collection

Function: `select_core_collection()`

Question: Which subset best represents genetic diversity?

`G` may be supplied as a relationship matrix or as a distance matrix, controlled by `type`. Relationship input is converted to square-root Euclidean distance. Before claiming a metric-distance guarantee, HapBlockR checks symmetry, non-negativity, zero diagonal and the triangle inequality.

Selection-strategy variants

strategy	Objective	Appropriate use
"maximin"	Maximise the minimum distance from the growing core	Protects against leaving a distinct accession unrepresented
"mean_distance"	Favour high average distance from the selected set	Seeks broad average dispersion but may protect extremes less strongly

`seed` controls the initial accession where the method requires one. `metric_tolerance` controls numerical validation, not biological similarity.

`merit` with `min_sel_value` applies an eligibility floor using the same "value", "percentile", "sd_above_mean" and "relaxed_pool" modes as truncation selection. Supply directional merit for which larger is better; reverse lower-is-better traits before setting the floor. The core objective remains diversity-first; the floor is not a hidden merit term.

Use this tool for germplasm banks, diversity panels and representative training populations. When the same subset will also serve as a crossing shortlist, combine the diversity objective with an appropriate merit floor or evaluate the retained accessions with a parent-selection tool.

4. Worked crossing decision

This synthetic example illustrates the decision record. Values are deliberately simple and are not universal thresholds.

4.1 Candidate evidence

Parent	GEBV	Reliability	Group	Female role	Male role	Flowering week
P01	12.4	0.78	A	yes	yes	6
P02	11.8	0.81	A	yes	yes	7
P03	10.9	0.72	B	yes	no	6
P04	10.5	0.69	B	no	yes	6
P05	9.7	0.44	C	yes	yes	9
P06	9.2	0.76	C	yes	yes	7

Pre-specified policy:

- reliability must be at least 0.60 for a recommendation;
- flowering windows may differ by no more than one week;
- P03 cannot be used as a male and P04 cannot be used as a female;
- no parent may appear in more than two planned crosses;
- at least one parent from groups A, B, and C must contribute; and
- the programme accepts genomic coancestry no greater than 0.18.

P05 is excluded from recommendations because reliability is 0.44. Its flowering week would also make most pairings infeasible. Both reasons are retained in the exclusion ledger.

4.2 Candidate cross results

Cross	Mean	Predicted SD	UC	10% downside	Reliability	Feasible
P01 x P03	11.65	1.30	12.72	9.98	0.72	yes
P01 x P04	11.45	1.52	12.70	9.50	0.69	yes
P02 x P04	11.15	1.48	12.36	9.25	0.69	yes
P03 x P06	10.05	1.72	11.46	7.85	0.72	yes
P02 x P06	10.50	1.10	11.40	9.09	0.76	yes
P01 x P05	11.05	1.90	12.61	8.62	0.44	no

UC is the usefulness criterion at the declared selection intensity. Reliability is conservatively the lower parental reliability in this illustration.

4.3 Decision

The highest feasible UC alone would favour P01 x P03. The full plan also needs group C and must respect parent capacity. The selected plan is:

Cross	Families	Reason
P01 x P03	40	Highest feasible UC; groups A and B
P02 x P04	30	Strong complementary A x B cross
P03 x P06	30	Adds group C and segregation potential

Quality-control gates pass, all parents meet the reliability threshold, role and flowering constraints pass, no parent occurs more than twice, all three groups contribute, and realised coancestry is 0.176. The coancestry ceiling is binding and should be monitored in the next cycle.

The recommendation is conditional on seed and pollen availability being confirmed before crossing. P02 x P06 is the first reserve cross.

5. Interpreting quality control and uncertainty

Every breeder-facing result should preserve:

- method and result-schema version;
- call and normalised parameters;
- random seed;
- immutable candidate and variant IDs;
- input SHA-256 hashes and transformation history;
- R, HapBlockR, dependency, and external-tool versions;
- quality-control gates and pass or fail status;
- warnings, fallbacks, exclusions, and failure reasons;
- decision and uncertainty tables; and
- validation status.

For Beagle phasing, also retain the executable path, Java version, JAR checksum, Beagle version, command, reference-panel and map checksums, input and output hashes, sample and allele-identity checks, observed-genotype concordance, and imputation rate.

Sensitivity analysis varies policy thresholds that could change the decision: reliability, effect significance, coancestry, parent capacity and selection intensity. The stability summary identifies parents and crosses that remain selected across scenarios and labels conditional recommendations with their appropriate alternative plans.

6. Interpreting results and defining their scope

HapBlockR converts the data, biological targets and operational constraints specified by the user into reproducible analytical and breeding recommendations. Their relevance is therefore driven principally by the quality and representativeness of the input data and by how faithfully the parameters express the breeding objective.

6.1 Choosing phased or dosage-based analysis

Phasing is a breeder and analyst choice determined by the biological question. Dosage data represent allele counts and support many relationship, prediction and block-level analyses without assigning alleles to parental chromosomes. Phased data add that chromosome assignment and are appropriate when the interpretation concerns transmitted gametes, cis relationships, recombination or within-block haplotype configurations.

HapBlockR accepts phased input and provides `phase_with_beagle()` for integrated Beagle phasing. It preserves executable and command provenance, checks sample and variant identity, REF and ALT alleles, variant order, observed-genotype concordance and imputation rate, and can calculate switch error and related accuracy measures when a phased truth set is supplied. Users should choose the route that matches their question and retain stable sample, variant and allele identities throughout the analysis.

6.2 Ploidy and allele representation

Dosage-centred relationship and marker-effect calculations support the ploidy values documented by their functions. The current hap1 and hap2 representations, Beagle integration, haplotype inference and compiled r-squared or rV-squared kernels describe diploid, biallelic data. Interpret results from those pathways within that declared representation.

6.3 Building relevance across families, cycles and environments

Prediction and selection results describe the population, environments, phenotypes, markers and model supplied by the user. HapBlockR provides several ways to make those inputs and decisions representative of the intended use:

- grouped and forward cross-validation evaluate prediction across families, populations, sites or cycles that match deployment;
- family- or genetic-cluster quotas retain representation across source groups;
- the replicated GA and its Optimal Haplotype Selection strategies identify parents from multiple groups whose favourable haplotypes complement one another;
- core-collection selection constructs diversity panels and representative training or reference populations;
- relationship-aware GA and optimum-contribution selection control coancestry; and
- multi-trait and genotype-by-environment models align selection with the target population of environments.

Consequently, the GA-selected diverse founder set can also form a representative training population for subsequent breeding cycles. The user defines the relevant families, groups, environments, objective weights and constraints; HapBlockR optimises and reports the decision within that specification.

6.4 From haplotype association to biological explanation

HapBlockR supports association evidence with population-structure and relatedness adjustment, multiple-testing control, cross-population harmonisation and comparison, epistasis analysis and fine mapping. These tools help distinguish reproducible trait-linked haplotypes from associations driven by confounding or sparse evidence.

A statistically supported and independently reproducible haplotype association can identify a chromosomal segment that harbours a causal gene or variant and can therefore provide a strong causal hypothesis. Association localises evidence to the tested haplotype segment; finer causal resolution is strengthened by fine mapping, gene annotation, functional evidence and independent validation. Reports should state whether the adjusted model succeeded and distinguish the associated segment from any specifically resolved causal variant.

6.5 Evidence density and sample support

HapBlockR reports the quantities needed to interpret the evidence: effective sample size, haplotype or allele frequency, missingness, numbers of environments or events, reliability and validation uncertainty. Users can therefore judge whether common and rare states are supported adequately for the intended decision. There is no single sample-size threshold for every trait, population and model; the relevant threshold is set from the design, effect size, frequency and required decision reliability.

7. Decision sign-off

Complete one copy for every promoted recommendation.

Field	Recorded value
Programme and cycle	
Target population of environments	
Data release and input hashes	
HapBlockR and R versions	
Method, parameters, and seed	
Training and validation design	
Accuracy and reliability gate	
Eligible candidates and exclusions	
Required and forbidden crosses	
Parent and period capacities	
Coancestry policy and realised value	
Binding constraints	
Selected plan and reserve plan	
Sensitivity result	
Scope and assumptions	
Analyst and date	
Breeder reviewer and date	
Data steward and date	
Final approval	

Sign-off confirms that the data, analytical evidence, assumptions and operational decision were reviewed and approved for the stated breeding use.

8. Supporting tools and complete function map

The nine decision tools depend on a wider evidence chain. This section explains the supporting tools and gives every exported function a defined place. Function-level help remains the authoritative source for every argument: use `?function_name` in R.

8.1 Genotype import and scalable backends

`read_geno()`

`read_geno()` standardises genotype sources as a `HapBlockR_backend`. Supported variants are:

format	Input	Operational note
"matrix"	In-memory individuals × variants dosage matrix	Supply matching <code>snp_info</code>
"numeric"	CSV or delimited text with SNP metadata and sample dosages	Declare sep and missing strings
"hapmap"	HapMap text	Check nucleotide and missing-call coding
"vcf"	VCF or compressed VCF	Both phased and unphased GT are read as dosage for this backend
"gds"	SNPrelate GDS	Preferred for repeated large-data access
"bed"	PLINK BED with BIM and FAM companions	Preserve all three files and sample order

For multiallelic VCF records, choose explicitly:

- "error": stop at the first unsupported assumption; preferred for a new formal analysis;
- "drop": exclude multiallelic records and record how many were lost; or
- "first_alt": retain the first alternative allele and treat other alternative indices as missing.

`clean_malformed = TRUE` removes records whose field count differs from the header. Use it only with an import report: cleaning a malformed source is a data transformation, not a harmless convenience.

For VCF and HapMap inputs, an SNPrelate GDS cache can be created and reused. The source and import options are protected by a SHA-256 manifest, so a changed source cannot silently reuse a stale cache.

Backend utilities

Function	Purpose	Breeder-facing check
<code>read_chunk()</code>	Read selected variants from a backend	Confirm returned sample and variant order
<code>close_backend()</code>	Release open backend resources	Call when a long workflow finishes
<code>read_geno_bigmemory()</code>	Use a file-backed genotype matrix	Retain descriptor and backing files together
<code>prepare_geno()</code>	Centre genotypes or whiten for LD adjustment	Record method and kinship decomposition
<code>get_V_inv_sqrt()</code>	Construct inverse square-root covariance transform	Inspect numerical regularisation

8.2 LD calculation and haplotype-block definition

Pairwise LD metrics

`compute_r2()` calculates conventional LD. `compute_rV2()` calculates relatedness- and structure-adjusted LD. Use:

- `r2` for a descriptive LD map when population structure is not expected to dominate the signal; and
- `rV2` when structure or close relatedness would inflate ordinary LD.

An adjusted statistic answers a different question, not merely a more accurate version of the same question. Record the kinship construction and compare both metrics during method development.

`Big_LD()` variants

`Big_LD()` detects blocks after MAF filtering and LD graph construction. Principal variants are:

Argument	Variants	Decision
<code>method</code>	"r2", "rV2"	Ordinary or structure-adjusted LD
<code>CLQmode</code>	"Density", "Maximal", "Louvain", "Leiden"	Clique-density or graph-community block definition
<code>split</code>	FALSE, TRUE	Keep or subdivide long regions
<code>appendrare</code>	FALSE, TRUE	Exclude or append rare markers according to the method
<code>singleton_as_block</code>	FALSE, TRUE	Exclude or retain isolated markers as one-marker blocks
<code>close_gaps_with_snps</code>	TRUE, FALSE	Close eligible inter-block gaps or leave them open
<code>checkLargest</code>	FALSE, TRUE	Apply the optional largest-block check

`CLQcut`, `clstgap`, `leng`, `subSegmSize`, `MAFcut`, `max_bp_distance` and `gap_k_rep` are method parameters rather than universal biological constants. Tune them in representative chromosomes and report sensitivity of block number, size and downstream prediction.

Compiled calculations accept one or two threads. More than two threads are rejected to preserve the package's tested execution envelope.

Genome-wide orchestration and diagnostics

Function	Purpose
<code>run_Big_LD_all_chr()</code>	Apply block detection chromosome by chromosome
<code>tune_LD_params()</code>	Compare candidate LD parameter settings
<code>compute_ld_decay()</code>	Summarise LD against physical distance
<code>plot_ld_decay()</code>	Plot LD decay and uncertainty
<code>plot_ld_blocks()</code>	Visualise detected blocks and marker positions
<code>summarise_blocks()</code>	Summarise block count, size and marker coverage
<code>scan_diversity_windows()</code>	Identify windows with unusual diversity
<code>run_ldx_pipeline()</code>	Orchestrate import, optional phasing, filtering, block detection, haplotype extraction and export

`run_ldx_pipeline()` supports:

- optional Beagle phasing;
- imputation by "mean_rounded", "mode" or "none";
- LD by "r2" or "rV2";
- the four CLQmode variants;
- numeric or character haplotype export;
- in-memory, streaming or bigmemory execution; and
- chromosome restriction for staged analyses.

The pipeline orchestrates the stages and retains their reports for review as one connected analytical record.

8.3 Phasing, haplotype extraction and harmonisation

External phasing

`phase_with_beagle()` launches an independently supplied Beagle 5 JAR. It checks Java and Beagle versions, limits threads to the tested range, records the executable, command and file hashes, and validates output sample, coordinate and allele identity.

Variants include:

- target-only phasing or reference-panel phasing through `ref_panel`;
- genetic-map or physical-position operation through `map_file`;
- chromosome-restricted operation through `chrom`;
- explicit Beagle burn-in, iteration, window and overlap controls; and
- truth-set validation through `truth_vcf`.

Without a truth set, observed-genotype concordance checks preservation of called dosage. With a phased truth set, the additional chromosome assignment supports switch-error measurement, and `assess_phasing_accuracy()` reports call rate, dosage accuracy, orientation-adjusted phased-allele concordance, and switch error by sample and chromosome. Threshold failures stop the formal phasing workflow and preserve the QC object.

`read_phased_vcf()` imports phased hap1 and hap2 matrices. Use this chromosome-resolved representation when the downstream interpretation depends on gamete transmission; dosage import remains available for allele-count analyses.

Haplotype construction variants

Function	Variants or role
<code>extract_haplotypes()</code>	Accepts a backend, phased hap1/hap2 list or dosage matrix; can restrict chromosomes and minimum block size
<code>infer_block_haplotypes()</code>	Infers block states when explicit phased haplotypes are unavailable; report this as inference, not validated phase
<code>collapse_haplotypes()</code>	Collapses rare or similar haplotype states according to its declared strategy
<code>decode_haplotype_strings()</code>	Converts encoded haplotype strings to interpretable states
<code>build_haplotype_feature_matrix()</code>	Builds model-ready haplotype indicators or dosages
<code>compute_haplotype_diversity()</code>	Calculates block-level diversity statistics
<code>compute_haplotype_grm()</code>	Constructs a haplotype relationship matrix

Function	Variants or role
<code>compute_dominance_grm()</code>	Constructs a dominance relationship matrix

Cross-study identity and comparison

Function	Purpose
<code>harmonize_haplotypes()</code>	Align block, marker and allele identities before comparison
<code>compare_haplotype_populations()</code>	Compare haplotype distributions between populations
<code>compare_block_effects()</code>	Compare effect direction and heterogeneity across block analyses
<code>compare_gwas_effects()</code>	Compare external GWAS effect evidence
<code>plot_block_funnel()</code>	Display cross-analysis effect consistency
<code>plot_haplotype_network()</code>	Display relationships among haplotype states

Never compare `block_12` from two analyses merely because the labels match. Coordinates, marker order, reference and alternative alleles, and haplotype encoding must be harmonised.

8.4 Haplotype association and candidate-region tools

Function	Purpose and important variants
<code>test_block_haplotypes()</code>	Tests haplotype or diplotype effects with declared structure and kinship adjustment
<code>estimate_diplotype_effects()</code>	Estimates the values of diplotype combinations
<code>decompose_block_effects()</code>	Separates block-level components for interpretation
<code>scan_block_epistasis()</code>	Screens pairwise block interactions
<code>scan_block_by_block_epistasis()</code>	Performs structured block-by-block scanning
<code>fine_map_epistasis_block()</code>	Resolves an interaction signal within a selected block
<code>rank_haplotype_blocks()</code>	Ranks blocks using declared evidence
<code>select_top_blocks()</code>	Applies a pre-declared top-block rule
<code>score_favorable_haplotypes()</code>	Scores candidate carriage of favourable states
<code>summarize_parent_haplotypes()</code>	Produces a breeder-facing parent × block summary
<code>define_qtl_regions()</code>	Groups evidence into candidate QTL regions
<code>integrate_gwas_haplotypes()</code>	Combines external GWAS and haplotype evidence
<code>export_candidate_regions()</code>	Writes candidate regions for downstream review

Association significance is not sufficient for selection. Review allele frequency, effective sample size, population structure, multiple-testing control, effect direction, uncertainty and replication. Epistasis scans require a pre-specified testing scope; searching all possible pairs and reporting only the largest interaction is not defensible.

8.5 Genomic prediction, validation and local effects

`run_haplotype_prediction()` variants

The function fits haplotype-aware prediction and returns whole-genome GEBV, local GEBV, block importance, relationship information, uncertainty and the standard result contract.

`marker_effect_method` supports:

Method	Interpretation
"gblup"	Kernel-based genomic BLUP
"rrblup"	Ridge-regression BLUP at feature level
"bayesb"	Bayesian variable selection with a point-mass-style sparse component
"bayesc"	Bayesian mixture shrinkage
"bayesa"	Bayesian heavy-tailed marker effects

The Bayesian variants require adequate MCMC iterations, burn-in and convergence review.

`include_dominance = TRUE` is supported only with the GBLUP route and requires BGLR. `importance_rule = "any", "all" or "mean"` controls how multi-trait block importance is aggregated.

`complete_decomposition` controls whether all marker effects are apportioned to local blocks.

Supporting prediction functions are:

Function	Purpose
<code>prepare_gblup_inputs()</code>	Align phenotypes, genotypes and IDs for GBLUP
<code>estimate_marker_effects()</code>	Estimate marker or feature effects using the declared model
<code>backsolve_snp_effects()</code>	Backsolve SNP effects from a genomic model
<code>compute_local_gebv()</code>	Aggregate effects into block-specific breeding values
<code>run_haplotype_stability()</code>	Assess stability of block importance across resampling or analyses

`cv_haplotype_prediction()` variants

validation	Use	Leakage controlled
"random"	Baseline interpolation within a reasonably exchangeable population	Row-level folds only
"grouped"	Predict new families, populations or other declared groups	Keeps complete groups out of training
"forward"	Predict future cycles, years or environments	Trains only on earlier observations

`n_rep` repeats the validation design and `k` controls folds where applicable. The output retains row-level out-of-fold predictions and calculates pooled accuracy and error metrics directly from them. Report those pooled metrics; fold-specific values remain available for heterogeneity assessment.

Choose grouped or forward validation when deployment concerns new families or future environments. Random validation estimates interpolation within a reasonably exchangeable population.

8.6 Multi-trait and environment-aware decisions

`build_selection_index()`

This function builds the internal multi-trait merit score from HapBlockR trait predictions. Trait directions are mandatory; units are optional metadata. Supply genetic and phenotypic covariance matrices with matching trait names. Four methods are available:

method	Objective and algorithm	Principal outputs
"smith_hazel"	Classical Smith-Hazel index using breeder-supplied economic weights	Linear coefficients, expected response, index accuracy and ranked merit score
"pesek_baker"	Deterministic Pesek-Baker desired-gain index	Linear coefficients, expected response and ranked merit score
"dgsi"	Desired-Gain Selection Index (DGSi) optimisation through DesiredGainR	Automatically selected best replicate, optimised coefficients, realised response, objective trace, rank correlation, coefficient stability and selected-set agreement
"qgsi"	Quadratic Genomic Selection Index (QGSi) through DesiredGainR	Linear, squared and cross-product score components, total candidate merit and candidate-specific contribution tables

Smith-Hazel and QGSi use `economic_weights`. Pesek-Baker and DGSi use `desired_gains`. Economic weights are policy quantities, not parameters to estimate from the same data merely to reproduce a preferred ranking. `directions` is the only sign convention: it declares increase or decrease, whereas weights and desired gains are non-negative magnitudes in the favourable-direction trait space. HapBlockR stops on negative objectives instead of silently changing their meaning.

DGSi is stochastic, but the breeder does not choose among runs manually. DesiredGainR executes `n_rep` independent searches, evaluates the complete desired-response objective and returns the replicate with the smallest objective automatically. The other replicates remain in compact diagnostics so that the breeder can assess convergence and stability. In threshold mode, trait thresholds define eligibility first; the optimised index then ranks eligible candidates and retains the requested number.

QGSi requires an explicit symmetric matrix of quadratic and cross-product weights. DesiredGainR does not fabricate this matrix from the desired-gain vector. Supply that matrix in the same favourable-direction trait space defined by `directions`. This prevents an automatically positive squared term from rewarding a large unfavourable deviation merely because it is large. QGSi contribution is candidate-specific. Therefore, HapBlockR reports each candidate's linear, squared and cross-product contributions and does not describe QGSi as one global additive marker-effect vector. A linear DGSi or Smith-Hazel index can, in contrast, propagate to a global index effect as the trait-weighted sum of the corresponding marker, haplotype or block effects.

DGSi's `realised_response` is a selected-set differential expressed in trait standard deviations after favourable-direction orientation. HapBlockR reports both the favourable-direction and original-direction forms and does not label either as an expected response in original units. QGSi receives `n_select`; HapBlockR calculates its model-expected per-trait gain with the total linear-plus-quadratic index variance. The QGSi trait table therefore reports `linear_weight`, not a fabricated global coefficient, and the complete quadratic-weight matrix remains available separately.

`fit_multitrait_gblup()`

The function fits correlated-trait GBLUP. Genetic and residual covariance matrices may be supplied or estimated by REML. Review covariance positive-definiteness, convergence, trait-specific reliability and the effect of missing target patterns. Prepared target precision modifies the residual variance. When a full named sampling covariance is supplied, the model retains its off-diagonal sampling relationships instead of reducing it to independent weights. By default, `sampling_covariance_mode = "sampling_only"` treats that matrix as the complete record-error covariance and does not fit an additional residual term. Use `"sampling_plus_residual"` only when the analysis requires a separate residual nugget beyond known sampling error. HapBlockR matches the actual genotype-trait-environment record keys, checks that the covariance diagonal agrees with normalised precision, and stops rather than silently substituting a zero covariance. Reported REML likelihoods use residual degrees of freedom, and PEV includes uncertainty from estimating fixed effects. Compare REML likelihoods only between models with the same fixed-effect design.

`build_environment_kernel()` and `fit_gxe_gblup()`

`build_environment_kernel()` constructs environmental similarity from standardised covariates.

`fit_gxe_gblup()` combines genomic and environmental kernels in a reaction-norm genotype-by-environment (G×E) model.

`environment_fixed = TRUE` estimates explicit environment means; `FALSE` changes their treatment. The package reports candidate performance and reliability by environment, supporting environment-specific parent selection. The separate across-environment table contains the internally estimated genomic main effect. HapBlockR does not combine environment-specific predictions with breeder-specified environment weights. If an approved external analysis has already produced an across-environment estimate, supply that estimate directly as an across-environment target. The G×E REML likelihood uses the rank of its fixed-effect design, and reported PEV includes fixed-effect estimation uncertainty. Compare REML likelihoods only between models with the same fixed-effect design.

8.7 Breeding-target preparation

`breeding_target_types()` returns the complete accepted-type definitions. `prepare_breeding_targets()` validates identity, estimand, estimation basis, uncertainty and favourable direction; calculates or normalises precision; deregresses random predictions; and returns the response used by internal models.

The principal variants are:

- fixed or model-adjusted estimates with `se_col`, `posterior_sd_col`, `precision_col` or a full named covariance;
- identity BLUPs with `reliability_col`, or `pev_col` plus `genetic_variance`;
- pedigree BLUPs with the same uncertainty plus `relationship_matrix = A`;
- fixed or random GCA with a declared `tester_population`;
- TGV with separate `additive_col` and `dominance_col`; and
- environment-specific targets with `environment_col`, which are routed to `fit_gxe_gblup()`.

The result records `value`, `deregressed_value`, `model_value`, `precision_weight`, reliability, PEV, direction, estimand and estimation basis for every genotype-trait-environment record. Hence, a breeder can trace exactly how an external estimate became an internal modelling response.

8.8 Operational mating feasibility

`screen_candidate_crosses()` filters candidate pairs before ranking. It can enforce:

- female and male role permission;
- fertility;
- overlapping flowering intervals;
- no selfing;
- different heterotic groups where required;
- quarantine compatibility;
- forbidden pairs; and
- directional reciprocal effects.

The result retains exclusion reasons. A reciprocal cross is not interchangeable when role permissions or reciprocal effects differ.

`certify_mating_plan()` audits the final plan. It checks minimum family size, required pairs, period-specific parent capacity, subpopulation quotas, repeated-cross policy and the same pairwise feasibility rules. Promotion requires a passing certificate; any unmet rule remains visible with its reason so the plan can be revised and certified again.

8.9 Stability, simulation and validation contracts

Function	Purpose
<code>assess_decision_stability()</code>	Compares selected sets and rankings across threshold, weight or scenario results
<code>ga_vs_ts_simulation()</code>	Compares realised multi-generation outcomes of founder and mating strategies
<code>plot_ga_vs_ts_simulation()</code>	Displays the simulated gain trajectories
<code>cluster_selection_groups()</code>	Quantifies representation of each selection strategy across retained genomic PCs
<code>plot_selection_clusters()</code>	Displays cluster coverage
<code>plot_parent_selection_pca()</code>	Shows selections in genomic or target-block feature space
<code>validate()</code>	Dispatches validation for supported package objects
<code>validate_hapblockr_result()</code>	Checks the common result schema, provenance and quality gates

`assess_decision_stability()` summarises Jaccard overlap, selection frequency and baseline agreement. It is the appropriate way to review many sensitivity runs; breeders should not compare long candidate lists manually.

Forward simulation estimates realised gain under the declared genetic architecture, recombination assumptions, selection intensity, founder set, mating rule and number of replicates. Report those assumptions with the result so breeders can compare scenarios on the same basis.

8.10 Metadata, exchange and output utilities

Function	Purpose
<code>validate_breeding_metadata()</code>	Validates required identifiers, vocabulary and referential integrity
<code>build_breeding_exchange()</code>	Constructs a versioned breeding exchange object

Function	Purpose
<code>write_breeding_exchange()</code>	Writes the exchange with provenance
<code>read_breeding_exchange()</code>	Reads and validates the exchange
<code>write_haplotype_numeric()</code>	Writes numeric haplotype features
<code>write_haplotype_character()</code>	Writes character haplotype states
<code>write_haplotype_diversity()</code>	Writes diversity summaries
<code>open_breeder_guide()</code>	Locates or opens the tagged PDF or editable Word edition

Every exchange preserves candidate, sample, variant, environment, study and analysis-run IDs. Display labels may change while the underlying identifiers remain immutable.

8.11 Recommended operational sequence

For a routine parent-and-cross decision:

1. Complete and approve the field-trial or genetic-evaluation analysis outside HapBlockR.
2. Prepare genotype-level targets with their estimand, estimation basis, direction, uncertainty and optional units.
3. Define whether the decision is across environments or environment-specific.
4. Import genotypes with an explicit multiallelic policy and verify identities.
5. Use phasing when the biological interpretation requires transmitted haplotypes; apply truth-set accuracy gates when truth data exist.
6. Detect and tune LD blocks; preserve block-definition provenance.
7. Fit prediction models and use a deployment-matched validation design.
8. Build the internal selection index and reliability gates.
9. Produce a truncation baseline.
10. Apply one appropriate alternative shortlist tool: GA, family quota or core collection.
11. Screen feasible crosses and calculate usefulness criterion (UC) using the most appropriate supported variance model available.
12. Use optimum contribution selection (OCS) when contributions and long-term coancestry are part of the programme objective.
13. Certify the mating plan, assess sensitivity and retain a reserve plan.
14. Validate the result contract and complete sign-off.

9. References

- Browning BL, Zhou Y, Browning SR (2018). A one-penny imputed genome from next-generation reference panels. *American Journal of Human Genetics* 103:338-348.
- Gao X, Starmer J, Martin ER (2008). A multiple testing correction method for genetic association studies using correlated single nucleotide polymorphisms. *Genetic Epidemiology* 32:361-369.
- Kim SA, Cho CS, Kim SR, Bull SB, Yoo YJ (2018). A new haplotype block detection method for dense genome sequencing data based on interval graph modelling of clusters of highly correlated SNPs. *Bioinformatics* 34:388-396.
- Mangin B and colleagues (2012). Novel measures of linkage disequilibrium that correct the bias due to population structure and relatedness. *Heredity* 108:285-291.
- Meuwissen THE (1997). Maximising the response of selection with a predefined rate of inbreeding. *Journal of Animal Science* 75:934-940.
- VanRaden PM (2008). Efficient methods to compute genomic predictions. *Journal of Dairy Science* 91:4414-4423.
- [HapBlockR project repository](#)
- [Official Beagle source and licence](#)

Change history

Edition 6, 26 July 2026: separated coverage-only `select_parents_ga()` from the explicit joint `select_parents_ga_ts()` tool; documented the joint coverage-and-whole-genome-merit objective, positive merit controls, eligibility variants, relationship controls, automatic replicated-search recommendation and distinct result provenance; and updated the decision-tool count, examples and function descriptions.

Edition 5, 25 July 2026: replaced field-trial analysis with a genotype-level breeding-target contract; defined adjusted mean, BLUE, identity BLUP, pedigree BLUP, breeding value, general combining ability and total genetic value; documented uncertainty, deregression and precision propagation; added `DesiredGainR` DGSi and QGSi methods; removed environment-weight aggregation; and renamed the breeder-facing broad-pool rule to `relaxed_pool`.

Edition 4, 25 July 2026: reframed result interpretation around data quality, population relevance and user-specified objectives; explained the package's phasing choice, population-representation, association-adjustment and fine-mapping support; defined OHS and OPV at first use; clarified automatic selection of the best feasible GA replicate; and added an editable, accessible Word edition alongside the tagged PDF.

Edition 3, 25 July 2026: expanded the guide into a practical package manual; documented all eight decision tools, their supported variants, required inputs, automatic recommendation rules, outputs and diagnostic review; clarified that `select_parents_ga()` automatically returns the best feasible replicate from its internal repeated searches; added genotype, phasing, LD, haplotype, association, prediction, multi-trait, G×E, target-input, feasibility, stability, metadata and exchange tool maps; and added a complete operational sequence.

Edition 2, 25 July 2026: added package compatibility, provenance and reliability gates, operational feasibility, a worked numerical decision, binding-constraint reporting, explicit diploid and phase limits, references, and the sign-off template; corrected the eight-tool count and optimum- contribution description.

Edition 1: initial plain-language guide.